

Text Mining

QSS20: Modern Statistical Computing

Recap of Regex

- Clean strings: `re.sub( )`
- Find all matches: `re.findall( )`
- Find any match: `re.search( )`
- Regex practice and regex cheatsheet.

Useful regex commands:

`\w \s \d .` alphanum, space, numeric, anything

`?` Previous character may occur

`+ | *` Match one or more

`( ) | (?:)` Matching versus non-matching group

Text as data!

- Can manipulate data and merge data frames– can finally do science!
- Supervised analyses
 - Keyword extraction
 - Parts-of-speech tagging
 - Named entity recognition
 - Sentiment analysis with a dictionary

Text as data!

- Can manipulate data and merge data frames– can finally do science!
- Supervised analyses
 - Keyword extraction
 - Parts-of-speech tagging
 - Named entity recognition
 - Sentiment analysis with a dictionary
- Unsupervised analysis
 - Bag-of-words representations
 - Topic modeling: Concept and implementation

Text as data! Pre-historic ChatGPT

- Can manipulate data and merge data frames– can finally do science!
- Supervised analyses
 - Keyword extraction
 - Parts-of-speech tagging
 - Named entity recognition
 - Sentiment analysis with a dictionary
- Unsupervised analysis
 - Bag-of-words representations
 - Topic modeling: Concept and implementation

Where to find data?

- Text documents:
 - News articles, legislation, Yelp reviews, song lyrics
 - Social media, complaints, online diaries.
- Multimodal
 - Audio to text (such as transcripts from camera data)

Code is a language as well

Bloomberg

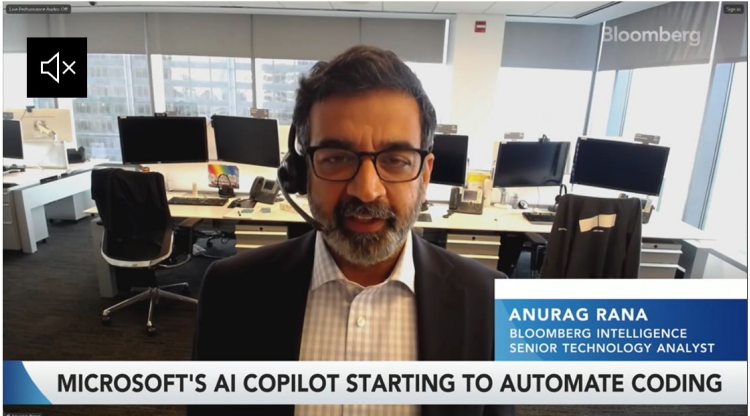
● Live TV Markets ▼ Economics Industries Tech Politics Businessweek Opinion More ▼

The AI Race: Meta Debuts Llama 3 | DOJ Scrutiny | Startups to Watch | AI Warfare | AI Unlocks Ancient Secrets | How AI Chatbots Work

Technology
AI

Microsoft's AI Copilot Is Starting to Automate the Coding Industry

The assistant is saving engineers hundreds of hours a month and is helping GitHub retain its edge over rivals including Amazon and Google.



ANURAG RANA
BLOOMBERG INTELLIGENCE
SENIOR TECHNOLOGY ANALYST

MICROSOFT'S AI COPILOT STARTING TO AUTOMATE CODING

Why Microsoft's AI Copilot Is a "Game Changer"

The Times Sues OpenAI and Microsoft Over A.I. Use of Copyrighted Work

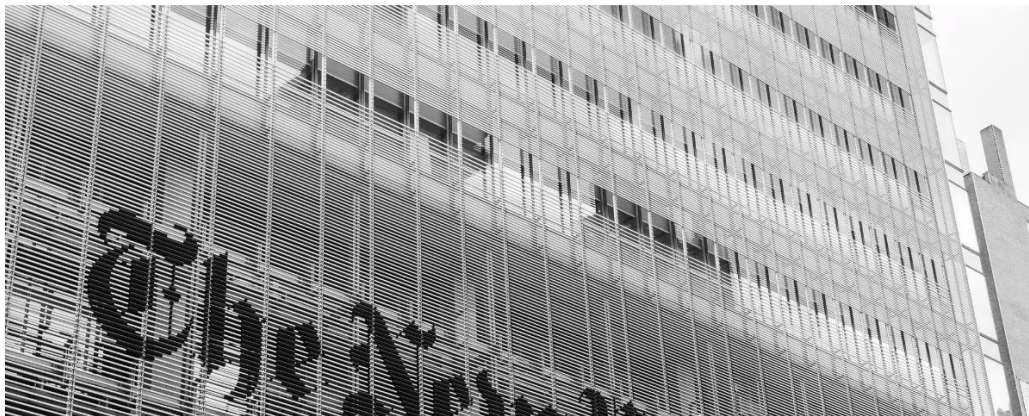
Millions of articles from The New York Times were used to train chatbots that now compete with it, the lawsuit said.



Share full article



1.3K

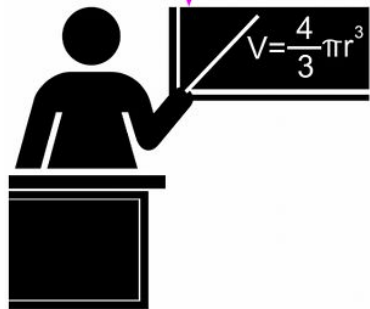


Where to find openly-available text?

- **Web scraping** or **API** to capture data (next module).
- [Kaggle text data](#)
DOJ press release; IMDB movie reviews
- [Restaurant reviews dataset](#)
- [NYC housing code violations](#)
- Facebook and previously Twitter
 - CrowdTangle

What do I mean by supervised?

Machine Learning



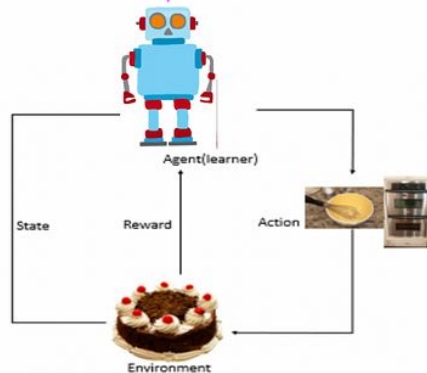
Supervised Learning

Humans teach



Unsupervised Learning

Machines figure it out



Reinforcement Learning

Machines interact with environment

Reinforcement Learning



QSS 45: AI and Machine Learning in Social Science

- Flavors of neural networks
 - Multiclass classification
 - Convolutional neural networks
 - Word2Vec
- Natural language processing with deep learning
 - Classical meets contemporary (LDA vs BERTopic)
- Zero-shot learning and labeling
- Fine-tuning and GPT workflows

Bigger, badder
neural nets



Any issues with this representation?

(WATER - WET) + FIRE = FLAMES

(PARIS - FRANCE) + ITALY = ROME

(WINTER - COLD) + SUMMER = WARM

(KING - MAN) + WOMAN = QUEEN

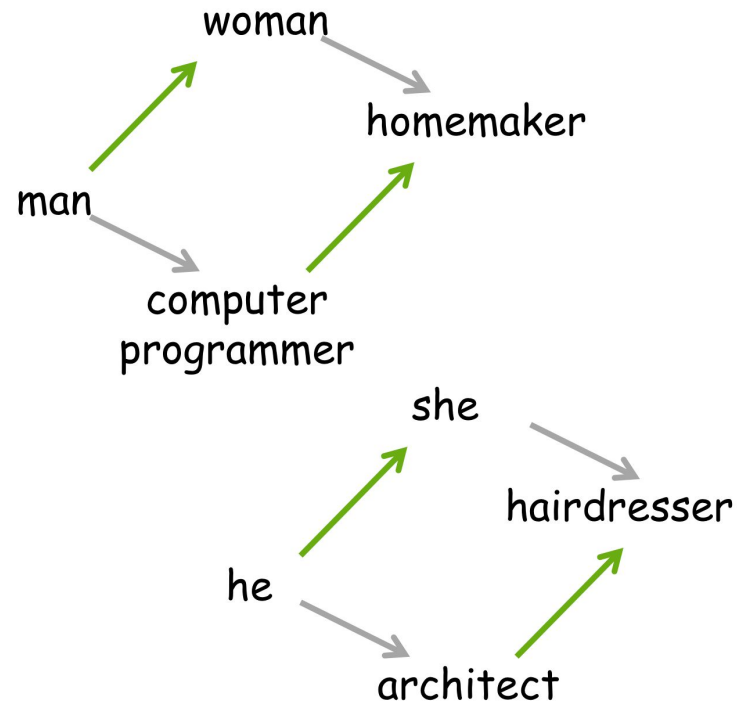
Any issues with this representation?

(WATER - WET) + FIRE = FLAMES

(PARIS - FRANCE) + ITALY = ROME

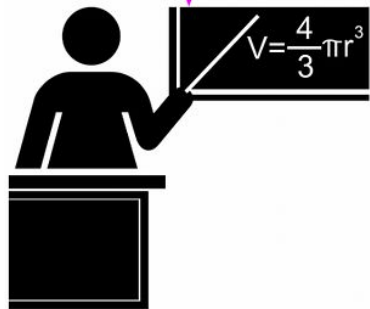
(WINTER - COLD) + SUMMER = WARM

(KING - MAN) + WOMAN = QUEEN



What do I mean by supervised?

Machine Learning



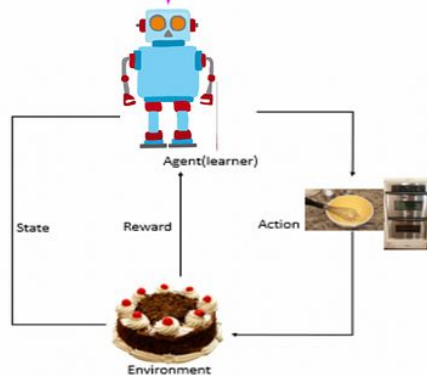
Supervised Learning

Humans teach



Unsupervised Learning

Machines figure it out



Reinforcement Learning

Machines interact with environment

What does supervised mean?

1. **Supervised Text Mining**

Specific pattern: Hashtags

Dictionary: List of words that represent a category or concept
`re.findall( r“civil rights | inequality | labor laws” )`

What does supervised mean?

1. **Supervised Text Mining**

Specific pattern: Hashtags

Dictionary: List of words that represent a category or concept
`re.findall( r“civil rights | inequality | labor laws” )`

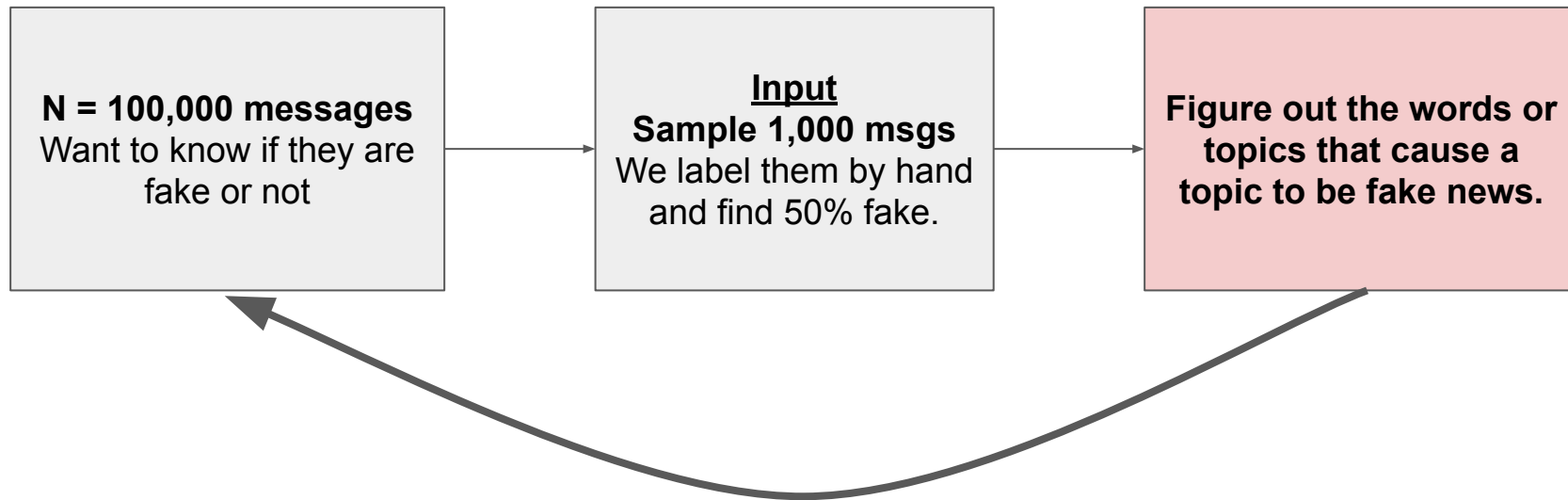
2. **Supervised Machine Learning**

Using some ML-algorithm, input text and output a category

I.e. WorkDay Application → accepted or not

I.e. Tweet → Supports mask wearing or not

Is this fake news?



Our example: NYC Airbnb Listing

name	neighbourhood_group	price
Nice and cozy little apt available	Bronx	75
Cozy and privat studio near Times Sq	Manhattan	140
NYCT02-3: Private Sunny Rm, NYU, Baruch, SOHO	Manhattan	75
Prime area in Chinatown and Little Italy	Manhattan	160
Midtown Manhattan Penthouse	Manhattan	100
2BR Comfy Apt - 15min from MIDTOWN	Queens	150
FourTwin bunkbeds- 5 minutes from JFK	Queens	90
Pvt Room in Quiet Home JFK 6mi LGA 10mi	Queens	38

Where the file is: QSS20_S25/public_data/airbnb_text.zip

What are interesting text features?

- **Positive words (sent.):** Nice, cozy, privat, comfy
- **Landmarks:** Little Italy, Chinatown, NYU, Soho, Time Square
- **Proximity to Airports:** JFK, LGA

name	neighbourhood_group	price
Nice and cozy little apt available	Bronx	75
Cozy and privat studio near Times Sq	Manhattan	140
NYCT02-3: Private Sunny Rm, NYU, Baruch, SOHO	Manhattan	75
Prime area in Chinatown and Little Italy	Manhattan	160
Midtown Manhattan Penthouse	Manhattan	100
2BR Comfy Apt - 15min from MIDTOWN	Queens	150
FourTwin bunkbeds- 5 minutes from JFK	Queens	90
Pvt Room in Quiet Home JFK 6mi LGA	Queens	38

How do we measure these elements?

Manual Coding | Version 1

```
## using the `name_upper` var, look at where reviews mention cozy
ab['is_cozy'] = np.where(ab.name_upper.str.contains("COZY"),
                        True, False)

## find the mean price by neighborhood and whether mentions cozy
mp = pd.DataFrame(ab.groupby(['is_cozy',
                              'neighbourhood_group'])['price'].mean())

## reshape to wide format so that each borough is row
## and one col is the mean price for listings that describe
## the place as cozy; other col is mean price for listings
## without that word
mp_wide = pd.pivot_table(mp, index = ['neighbourhood_group'],
                          columns = ['is_cozy'])

mp_wide.columns = ['no_mention_cozy', 'mention_cozy']
```

Results: What do you observe?

neighbourhood_group	no_mention_cozy	mention_cozy
Bronx	89.231088	74.214286
Brooklyn	128.175441	91.130224
Manhattan	204.109775	129.917140
Queens	102.596682	80.344388
Staten Island	120.650307	74.319149

Results: What do you observe?

neighbourhood_group	no_mention_cozy	mention_cozy
Bronx	89.231088	74.214286
Brooklyn	128.175441	91.130224
Manhattan	204.109775	129.917140
Queens	102.596682	80.344388
Staten Island	120.650307	74.319149



Cozy BNBs are
cheaper!

Manual Coding | Version 2

```
## construct dictionary
space_indicators = {'small': ['COZY', 'COMFY', 'LITTLE', 'SMALL'],
                    'large': ['SPACIOUS', 'LARGE', 'HUGE', 'GIANT']}

## for each listing, find the number of occurrences
## of words in each key

### first, let's test with one listing
practice_listing = "NICE AND COZY LITTLE APT AVAILABLE"
```

Count number of appearances

```
### example string
practice_listing = "NICE AND COZY LITTLE APT AVAILABLE"

### splitting at space and looking at overlap with each key in the
dictionary
words_overlap_small = [word for word in practice_listing.split(" ")
                        if word in space_indicators['small']]

words_overlap_large = [word for word in practice_listing.split(" ")
                       if word in space_indicators['large']]

### could then take length as a fraction of all words
len(words_overlap_small)/len(practice_listing.split(" "))
len(words_overlap_large)/len(practice_listing.split(" "))
```

Automated Process 1 | Parts-of-speech tagging

- Very manual, not very rigorous.
 - Can we automated these processes?
- Also did not capture other attributes!
- Entities
 - **Places:** Chinatown, Little Italy, Times Square
 - **Infrastructure:** LGA, JFK

Parts-of-Speech Tagging | Example

```
## specify example
example_for_tag = "This is a chill apt next to the subway in LES
                  Chinatown"

## try part of speech tagging using nltk
tokens = word_tokenize(example_for_tag) # Generate tokens
tokens_pos = pos_tag(tokens) # generate part of speech tags for
                              those tokens
```

Output is list of tuples with:
(Word, Part of Speech)

```
for one_tok in tokens_pos:
    print(one_tok)
```

```
('This', 'DT')
('is', 'VBZ')
('a', 'DT')
('chill', 'NN')
('apt', 'JJ')
('next', 'JJ')
('to', 'TO')
('the', 'DT')
('subway', 'NN')
('in', 'IN')
('LES', 'NNP')
('Chinatown', 'NNP')
```

Common parts of speech

“This is a chill apt next to the subway in LES Chinatown”

tag	meaning	in our example
CC	coordinating conjunction	
CD	cardinal digit	
DT	determiner	This; the; a
JJ	adjective	apt; next
JJR	adjective (comparative; e.g., bigger)	
NN	noun (singular; e.g., desk)	chill; subway
NNS	noun (plural; e.g., desks)	
NNP	proper noun (singular; e.g., Harrison)	LES; Chinatown
NNPS	proper noun (plural; e.g., Americans)	
TO	to	
UH	interjection	
VB	verb (base form; e.g., take)	
VBD	verb (past form; e.g., took)	
VBG	verb (gerund/present; e.g., taking)	
VBN	verb (past participle; e.g., taken)	
VBZ	verb (3rd person singular present; e.g. takes)	

Targeting a particular type of word

If we are interested in proper nouns (i.e. Chinatown), then we can use a list comp.

One_tok := (word, POS)

```
all_prop_noun = [one_tok[0] for one_tok in tokens_pos  
                  if one_tok[1] == "NNP"]  
all_prop_noun
```

```
all_prop_noun
```

```
['LES', 'Chinatown']
```

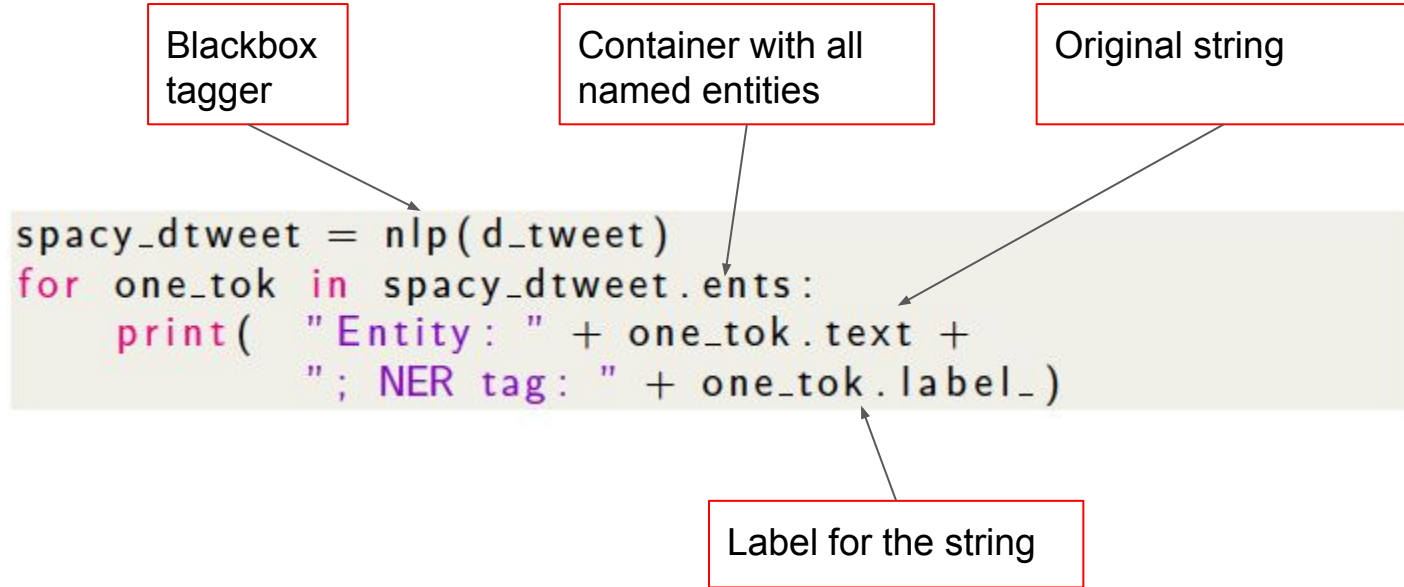
Automated Process 2 | Named Entity Recognition (NER)

- **Proper noun is broad:**
Chinatown and **Charles Dickens** are both proper nouns.
- **Granular subtypes** using spaCy label schemes
 - PERSON: President Sian Beilock
 - FAC: Buildings or bridges— Boston Logan International Airport
 - GPE: Cities and states— Hanover, NH
 - ORG: Organizations— Dartmouth College
 - DATE: Oct. 10, 2023
- Switch from nltk to spaCy

Example Tweet: What will be labeled?

We'll be hosting on-campus COVID-19 booster clinics at Dartmouth College in New Hampshire from 9 a.m. to 6 p.m. on Monday, Jan. 10, and Tuesday, Jan. 11, at Alumni Hall in the Hopkins Center. For information on how to register and additional winter updates, head to

Named Entity Extraction



NER output

We'll be hosting on-campus COVID-19 booster clinics at Dartmouth College in New Hampshire from 9 a.m. to 6 p.m. on Monday, Jan. 10, and Tuesday, Jan. 11, at Alumni Hall in the Hopkins Center. For information on how to register and additional winter updates, head to

Entity: Dartmouth College; NER tag: ORG
Entity: New Hampshire; NER tag: GPE
Entity: 9 a.m. to 6 p.m.; NER tag: TIME
Entity: Monday, Jan. 10; NER tag: DATE
Entity: Tuesday, Jan. 11; NER tag: DATE
Entity: Alumni Hall; NER tag: WORK_OF_ART
Entity: the Hopkins Center; NER tag: FAC

Notebook for today

https://github.com/herbertfreeze/QSS20-X26/blob/main/activities/05_textasdata_partI_textmining.ipynb

If you have problems installing... Virtual envs

In terminal:

```
conda create -n qss20 python=3.10
```

```
conda activate qss20
```

```
pip install -U spacy
```

```
pip install jupyter
```

```
python3 -m spacy download en_core_web_sm
```

jupyter notebook – your notebook should now run

Notebooks Workflow

- On GitHub: Press **Sync** and **Update**
- On GitHub Desktop
 - **Fetch and pull**: This updates your codebase on your computer
 - Make a copy of the notebook in your folder and work on it there
- Before you leave class today, on GitHub Desktop, **commit** and **push**

Section one of notebook

Task: Play around with the Dartmouth tweet and see results

- What happens when you abbreviate New Hampshire?
- What happens when you add Pfizer before COVID-19?
- What entities seem misclassified?

Sentiment Analysis: Dictionary-based approach

Words are given a score between -1 and 1.

In basic form, dictionary assigns word with a score in this range.

- Love: 1
- Hate: -1
- Ambivalent: 0

VADER Sentiment Scoring Code

Initialize the sentiment scoring object

```
## initialize a scorer  
sent_obj = SentimentIntensityAnalyzer()  
  
## score one listing  
practice_listing = "NICE AND COZY LITTLE APT AVAILABLE"  
sentiment_example = sent_obj.polarity_scores(practice_listing)
```

Apply the polarity_score method on
the practice listing

Sample output

```
print("String: " + practice_listing + " scored as:\n" + str(sentiment_example))
print("String: " + practice_listing_2 + " scored as:\n" + str(sentiment_example_2))
print("String: " + practice_listing_3 + " scored as:\n" + str(sentiment_example_3))
```

String: NICE AND COZY LITTLE APT AVAILABLE scored as:

{'neg': 0.0, 'neu': 0.641, 'pos': 0.359, 'compound': 0.4215}

String: NICE AND COZY LITTLE APT AVAILABLE. REALLY TERRIBLE VIEW. scored as:

{'neg': 0.257, 'neu': 0.531, 'pos': 0.212, 'compound': -0.1513}

String: NICE AND COZY LITTLE APT AVAILABLE. HAS RATS THOUGH. scored as:

{'neg': 0.0, 'neu': 0.741, 'pos': 0.259, 'compound': 0.4215}

Compound is summary of first three between -1 to 1

Many words classified as neutral

Terrible decreases score, but rats is context specific.

To improve: Augment the default VADER dictionary

```
## create a dictionary with
## negative scores for pests
pest_words = {
    'rat': -1.9, 'rats': -1.9,
    'mice': -1.9, 'mouse': -1.9,
    'roach': -1.9, 'cockroach': -1.9
}
## update the lexicon with that
## dictionary (created new sentiment object
## to avoid writing over older one)
new_si = SentimentIntensityAnalyzer()
new_si.lexicon.update(pest_words)
## use that updated scorer
new_si.polarity_scores(practice_listing_3)
```

Score decreased to 0.228

```
print("After lexicon update: " + practice_listing_3 + " scored as:\n" + \
      str(new_si.polarity_scores(practice_listing_3)))
```

```
After lexicon update: NICE AND COZY LITTLE APT AVAILABLE. HAS RATS THOUGH. scored as:
{'neg': 0.228, 'neu': 0.551, 'pos': 0.22, 'compound': -0.0258}
```

Milestone One Link

QSS Honors Theses

Will chatting with AI make you go to war (Tracy Weener)

- Deploys large-scale AI chatbots in Taiwan in survey experiment
- Defense policy and nuclear energy



How does AI-generated voices impact trustworthiness (Malachai Ah Yo)

- Generates 60 voices and deploys them in a survey experiment
- Effects of gender, age, and income



Sustainable practices of Upper Valley Farmers (Ben Shaman)

- Surveys 100 upper valley farms
- Including interviews



QSS Honors Theses start next week

Primed and Prejudiced: AI versus humans biases in disseminating political content

Maya Magee

- Compares 2,200 human responses with AI responses during the 2024 elections
- Simulates information spread across social networks with millions of users



Regional Attitudes towards Renewable Energy Technologies in the Era of Misinformation

Will Ermarth

- Large scale survey on renewable energy across 2,500 respondents & social media
- Uses AI generated misinformation and measures impact on investment support

